PART - 3

ARRAYS

Array is a User defined Non-primitive data type which is represented by [ ].

Array is a continuos block of memory which is used to store similar type of values.

(OR)

Array is a contiguess block of memory which is used to store homogenous values.

SYNTAX TO CREATE REFERENCE VARIABLE FOR ARRAY:

```
Datatype[]   identifier;
```

or

```
Datatype  [] identifier;
```

or

```
Datatype  identifier[];
```

Eg: 1) int[] a; → 'a' is a reference variable which is of type integer array

2) char[] ch; → ch is a reference var of type character ar

3) String[] s; → s is a reference var of type String array

4) Object[] obj; obj is a reference var of type Object array

5) double[] b; → b is a reference var of type 'double array.

NOTE: Inside array reference variables which can store only two thing either null or reference of array object

Eg: int a[] = 10; // CTE

int[] a = null; // CTS

String[] s = "Rajesh"; // CTE

String[] s = null; // CTS

The default value of array reference variable of any t is always null.

Program to find default value of Array reference vari

```
class A1
{
 static int[] a;
 int[] a;
 public static void main (string [] args)
 {
 ~~sys~~ A1 obj = new A1();
     System.out.println (a);
     System.out.println (obj.a);
 }
}
```

To create an Object of array:

In java we can create an object of array in 2 ways: They are:

1) With the help of declare & initialization statement

2) With the help of new operator.

WITH THE HELP OF DECLARE & INITIALIZATION STATEMENT :-

SYNTAX:

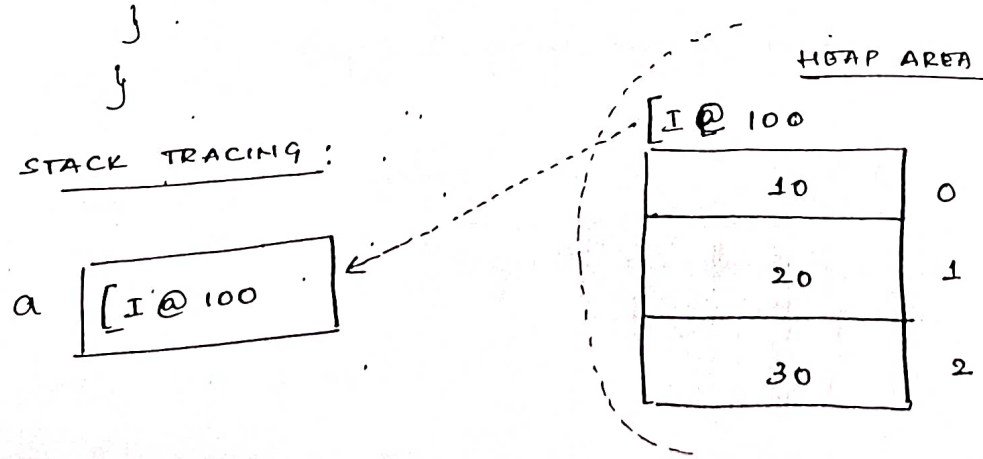
Datatype [] identifier = { value1, value2, value3, ... valuen }

or

Datatype [] identifier = { element1, element2, element3, ... elementn }

Eg: class A3
{
public static void main (String [] args)
{
int[] a = { 10, 20, 30};
System.out.println(a);
}
}

O/P:
[I@.....



NOTE: We cannot initialize the array reference variable seperately by using initialization statement. we get Compile Time Error.

```
class A5 :
{
public static void main (String[] args)
 {
    String[] s;
    s = {"Sheela", "Leela", "Mala", "Laila"}; //CTE
 }
}
```

NOTE: In the above examples, the size of the array will be equal to number of elements passed within curly braces.

Can we create an Empty Array Object?

Yes, we can create an empty array object ie an Array object with the size 0

Eg: class A6

```
{
 public static void main(String[] args)
 {
    int[] a = { };
    System.out.println(a.length);
 }
}
```

o/p:
0

length :

length is a non-static variable of Array.

It gives us the size of array object ie total numbe of elements present inside array object.

NOTE: Address for the array memory location will always be assigned based on:

0) Dimension of Array created

↳ 1) Single / one Dimension - [

2) Two Dimension - [[

3) Three Dimension - [[[

2) Type of the array declared

Eg: int[] a = {10, 20, 30};

System.out.println(a);

Op: [I@ some hexadecimal value.

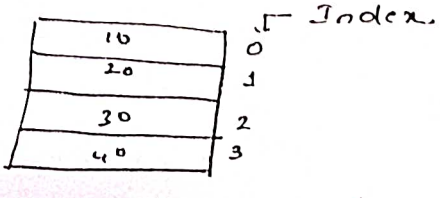
double[] a = {10, 20};

System.out.println(a);

Op: [D@ some hexadecimal value.

NOTE: Each and every blocks of the memory will be indexed from 0 to n-1 where 'n' is the no of values we pass during array object creation.

int[] a = {10, 20, 30, 40, 50}



WITH THE HELP OF NEW OPERATOR:

We can create an object of array with the help of new operator using following syntax:

```
new Datatype [size];
```

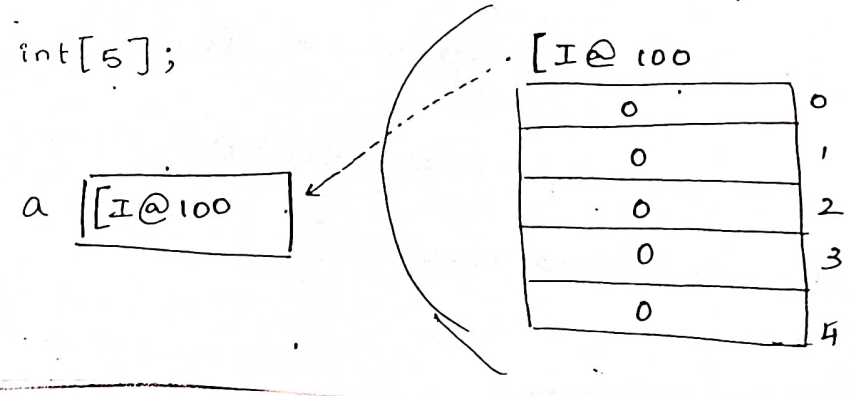
Whenever we are using new operator, it is mandatory for us to provide the size of the array. Else we get Compile Time Error.

Eg:
```
class A10
{
public static void main (String [] args)
{
  int [] a = new int [ ]; //CTE → missing dimension.
  System.out.println (a);
}
}
```

NOTE: The new operator will create the array object & returns its address.

The array object created will be initialized with the default values of data types used.

Eg: `int a[] = new int[5];`



TO ACCESS THE ELEMENTS OF ARRAY:

To access the elements of Array we make use of index of array objects.

Index always starts from 0 and ends at (a.length-1 [size-1].

Index will always be of type int.

We can access the elements of array using below syntax

referencevariable [Index value]

Eg: class Ag
{
public static void main (String [] args)
{
int a[] = new int [5];
a[0] = 10;
a[1] = 20;
a[2] = 30;
a[3] = 40;
a[4] = 50;
for (int i = 0; i < a.length; i++)
{
System.out.println(a[i]);
}
}
}

O/P:
10
20
30

▶ ArrayIndexOutOfBoundsException:

We get ArrayIndexOutOfBoundsException if we try to acce the Index which is either less than 0 or greater than or equal to size of the array.

Eg:

```
class All
{
 public static void main (String[] args)
 {
  int[] a = new int[3];
  System.out.println [a[3]); // ArrayIndexOutOfBounds Exception
  System.out.println(a[-1]); //      "
 }
}
```

WAJP to read 'n' no of elements from the user & store it & then print it.

```
import java.util.Scanner;

class ReadAndPrint
{
    public static void main (String [] args) {
 Systam. Scanner s = new Scanner (System.in);

  System.out.println ("Enter the total number of elements");

   int i = s.nextInt();

   int[] a = new int[i];

   System.out.println("Enter the first element
```

```
for (int n=0 ; n<i ; n++)
{
  System.out.println("Enter " + (n+1) + " element");
  a[n] = s.nextInt();
}
for (int n=0 , n<i , n++)
{
  System.out.println(a[n]);
}
}
```

WAJP to find sum of all the elements of an array.

```
import java.util.Scanner;
class sum
{
 public static void main (String[] args)
 {
  Scanner s = new Scanner(System.in);
  int sum=0;
  System.out.println("Enter total number of elements");
  int size = s.nextInt();
  int[] a = new int[size];   } or int[] a = new int[s.nextInt()];
  for (int i=0 ; i< size ; i++)
  {
   System.out.println("Enter" + (i+1) + "Element");
   a[i] = s.nextInt();
  }
  for( int i=0 ; i<size ; i++)
  {
   sum = sum + a[i];
  }
  System.out.println("Sum of all the elements is :" + sum);
 }
```

3. WAJP to find sum of even index elements.

```
import java.util.Scanner;

class EvenSum
{
  public static void main(String[] args)
  {
    int sum=0;
    Scanner s= new Scanner(System.in);
    System.out.println("Enter total number of elements")
    int[] a= new int[s.nextInt()];
    for(int i=0; i<a.length; i++)
    {
      System.out.println("Enter"+(i+1)+" element");
      a[i]=s.nextInt();
    }
    for(int i=0; i<a.length; i++)
    {
      if(i%2==0)
        sum+=a[i];
    }
    System.out.println("The sum of even index elements is
                        +sum);
  }
}
```

WAJP to find sum of odd index elements of an array.

```
import java.util.Scanner

class OddSum
{
 public static void main(String [] args)
  {
   int sum=0;
   Scanner s= new Scanner(System.in);
   System.out.println("Enter total number of elements");
   int[] a = new int[s.nextInt()];
   for(int i=0; i<a.length; i++)
    {
     System.out.println("Enter " + (i+1)+ " element");
     a[i]= s.nextInt();
    }
    for(int i=0; i<a.length; i++)
     {
        if(i%2!=0)
            sum+= a[i]
     }.
    System.out.println("The sum of odd index elements
                        is :" + sum);

  }
}
```

SORTING :

Q. WAJP to find largest element of the array. (by default consider 1st no as the largest & write the logic)

```
class Large
{
  public static void main (String [] args)
  {
    int [] a = {30, 50, 70, 10};

    int largest = a[0];

    for (int i=1; i< a.length; i++)
    {
      if (a[i] > largest)
      {
        largest = a[i];
      }
    }
    System.out.println("The largest element of the array is:
                        + largest);
  }
}
```

Q. WAJP to swap two variables.

```
class Swap
{
 public static void main (String [] args)
 {
   int a = 10;
   Int b = 20;
   Int temp = 0;

   a = temp;
   b = a;
```

```
    b= temp;
    S.o.pln (a);
    S.o.pln (b);
  }
}
```

2. WAJP to swap 2 variables without using extra variable

```
class Swap
{
  public static void main (String [] args)
  {
    int a=10;
    int b=20;
    a= a+b;
    b= a-b;
    a= a-b;
    S.o.pln (a);
    S.o.pln (b);
  }
}
```

Sorting is a design technique which is used to arrange the elements in proper order. ie either in ascending order or descending order.

In java, we have different type of sorting algorithm:

i) Bubble sort

ii) Selection sort

iii) insertion sort

iv) Heap sort

v) Merge sort

Sorting can be done in 2 ways:

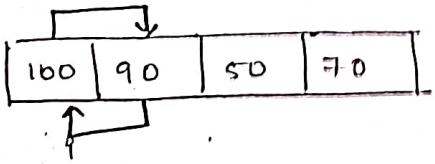
i) with the help of algorithm/ logic

ii) with the help of built-in

BUBBLE SORT ALGORITHM:

* If an array has 'n' no of elements, bubble sort technique will have n-1 no of pass.
* In each pass largest element of the array swaps continously until it moves to its original position
* The above thing (swapping) is acchieved by comparing an element with the next element of the element is largest, it is swapped immediately with the next element as well as the pointer is also shifted to next
* If the element is not larger than the next element it'll not swap the element only the pointer is shifted.

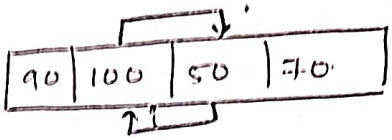
g: Pass 1 :



largest = 100

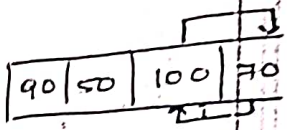
100 > 90 → T

swap the element ⇒



100 > 50 ⇒ True

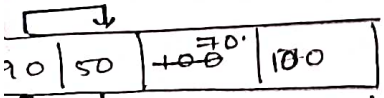
swap the element ⇒



100 > 70 → True

Swap the element

Pass 2 :



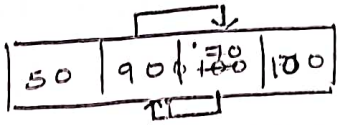
sorted

unsorted

largest = 90

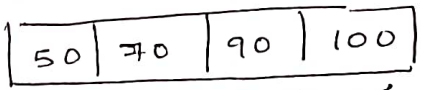
90 > 50 → True

swap the elements ⇒



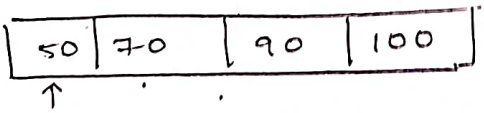
90 > 70 → True

Swap the elements ⇒



unsorted sorted

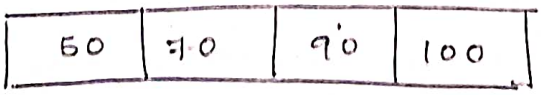
PASS 3 :



largest = 50;

50 > 70 → false

don't swap but shift the pointer.



```
class BubbleSort
{
public static void main (String[] args)
 {
  int[] a = { 100, 90, 50, 70 };
  // to sort using bubble sort technique
  for (int i=0; i< a.length-1 ; i++)
  {
    // for comparison & swapping
    for( int j=0; j< a.length-1-i; j++)
     {
        if ( a[j] > a[j+1])
        {
           int temp= a[j];
           a[j]= a[j+1];
           a[j+1]= temp;
        }
     }
  }
  S.o.println ("The elements of array after sorting :");
  for(int i=0; i<a.length; i++)
  {
     S.o.println (a[i]);
  }
 }
}
```

The program to bubble sort in descending order:

just change `a[j] < a[j+1]`

SELECTION SORT:

If an array has 'n' no. of elements then selection sort will have (n-1) no of passes.

ALGORITHM FOR ASCENDING ORDER:

i) In each pass we try to identify the position of smallest element and it'll be swapped to its original position.

position = i

j = i+1

compare a[position] with a[j] upto length of array.

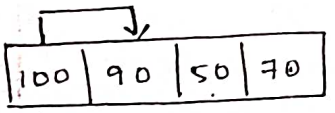
If a[position] > a[j] then assign the value of 'j' to position variable

ii) After all the comparisons, in each pass check whether the position variable is still having the value which is same as that of initialized value.

If it is not same, only then we should go for swapping of elements.

iii) In the worst scenario, selection sort will have maximum of only one swapping for each pass.

Eg: PASS-1.

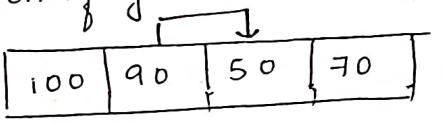


pos = 0

j = 1

100 > 90 → True

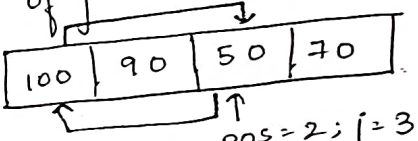
change the position of j

pos = j ⟹ 

pos = 1 ; j = 2

90 > 50

change position of j

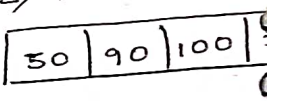
pos = j ⟹ 

pos = 2 ; j = 3

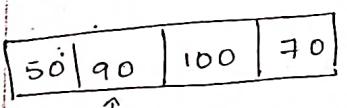
50 > 70 ⇒ False

Smallest = 50

if (pos != 0) ⇒ True

swap ⇒ 

PASS 2:

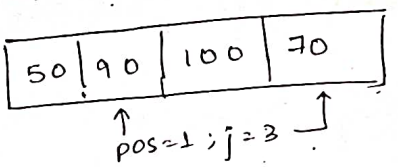


pos = 1 ; j = 2

90 > 100 ⇒ False

(pos != 1) ⇒ False

No changes requidered.

& updata j ⇒ 

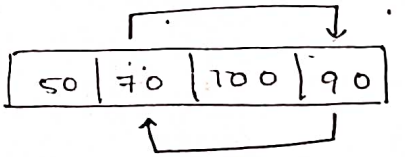
pos = 1 ; j = 3

90 > 70 ⇒ ~~False~~ True.

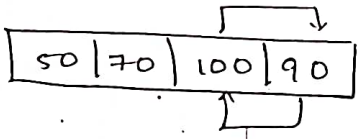
change the position to j

pos = j ; // 3

if (pos != 1)

swap ⇒ 

PASS 3 :

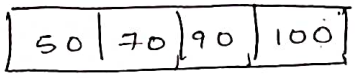


pos = 2 ; j = 3

100 > 90 → True → change the pos to j

pos = j // 3

is (pos != 2)

swap ⇒ 


```
class SelectionSort
{
  public static void main (String [] args.)
  {
    int [] a = { 100, 90, 50, 70, 40, 60 };
    // selection sort logic
    for (int i=0; i<a.length; i++)
    {
      int pos = i;
    for (int j=0; j<a.length; j++)
      {
        if (a[pos] > a[j])
        {
          pos = j;
        }
      }
      if (pos = j)
      {
        int temp = a[i];
        a[i] = a[pos];
```

```
            a[pos] = temp;
        }
    }
    for(int i=0; i<a.length; i++)
    {
        System.out.println(a[i]);
    }
}
}
```

2) with the help of built-in:

We can sort the arrays with the help of built-in method sort method which is present inside arrays class

Sort method is an overloaded method which can accept any type of arrays or array reference.

It is a static method which can be called with the help of its class name as below:

Arrays.sort (array reference);

Arrays is a built-in class present inside util package.

Eg :

```
import java.util.Arrays;
class A1
{
  public static void main (String [] args)
  {
   int[] a = { 100, 90, 50, 70, 40, 60};
   // to sort array. using built-in
   Arrays.sort (a);
   S.o.pln ("The sorted arrays are:");
   for (int i=0; i<a.length; i++)
   {
    S.o.pln (a[i]);
   }
  }
}
```

▶ TO SORT OBJECTS INSIDE ARRAYS :

In java, if we need to sort the objects, they must be of comparable type. If the objects are not of comparable type then we get ClassCastException.

Eg :

```
class Student
{
  int sid;
  String name;
  Student() { } :
```

```
Student ( int sid, String name)
{
this.sid = sid;
this.name = name;
}

public void displaydetails ( )
{
System.out.println ("student id :"+sid);
System.out.println ( "student name : " + name);
System.out.println ("= = = =
```

```
    }
    //to sort the elements using built-in.
    Arrays.sort(s);    // Class Cast Exception.
    for(int i=0; i<s.length; i++)
    {
        System.out.println(s[i]);
    }
  }
}
```

To overcome the problem of Class cast Exception, the object must be of comparable type.

Steps to make an object comparable type :

STEP 1 : implement an Interface called as Comparable interface which is present inside java.lang package.

STEP 2 : Override the compareTo() method which is present inside Comparable Interface as below :-

1) compare the current object state with the passed object state using >, < or == operators & return an int value based on comparison.

2) The int value can be returned as below :

i) If the current object state is greater than passed object state, then we must return a positive integer.

ii) If the current object state is lesser than passed object state then we must return -ve integer.

iii) If the current object state is ~~equal to~~ same as that of passed object state the we must return 'zero'.

Eg:

```
class Student implements Comparable
{
int sid;
String name;
Student() { }
Student(int sid, String name)
{
this.sid = sid;
this.name = name;
}
public void displayDetails()
{
System.out.println("student id :" + sid);
System.out.println("student name :" + name);
System.out.println("==========");
}
public String toString()
{
  return sid + ":" + name + " ";
}
//to override the compareTo(Object o)
public int compareTo(Object o)
{
  if (this.sid > ((Student)o).sid)
  {
   return 1;
  }
```

```
else if (this.sid<((student)o).sid)
  {
    return -1;
  }
else

    return 0;
 }
}

class A2
{
 public static void main (String[] args)
  {
    student[] s = new student.[5];
    s[0] = new student (101, "sheela");
    s[1] = new student (104, "Leela");
    s[2] = new student (105, "Laila");
    s[3] = new student (103, "Urmila");
    s[4] = .new student (102, "Malo");
   System.out.println ("Before sorting:" );
   for( int i=0; i< s.length; i++)
    {
      System.out.println (s[i]);
    }
    Arrays.sort(s);
    System.out.println ("After sorting:");
    for(int i=0; i< s.length; i++)
    {
      System.out.println (s[i]);
    }
  }
}
```